

=====

DAY -3 -

=====

Transferring Files from Jenkins to Remote Server

=====

1. Setup connection from Jenkins to remote server
2. Install SSH plugins
3. Create a job to copy files to remote server

Have 2 Servers

1. Jenkins Server
2. Test Server (Amazon Linux 2)

In Jenkins Server

ssh-keygen

eval \$(ssh-agent -s) --- to start the ssh agent

vi MyKey.pem

copy paste pem data

chmod 400 MyKey.pem

ssh-add MyKey.pem

cd /root/.ssh/

ssh-copy-id -i id\_rsa.pub ec2-user@172.31.22.33

Testing to connect = ssh ec2-user@172.31.22.33

install ssh plugins

-----

Manage Jenkins --> Plugins --> Publish Over SSH

Manage Jenkins --> System --> Publish Over SSH

cd /root/.ssh

cat id\_rsa

--> Publish over SSH --> key -->

Paste this id\_rsa data in "Key" section in Jenkins

SSH Server --> ADD

Name = TestServer

Hostname = Private IP

Username = ec2-user

Test Configuration

Create a new job

-----

New item --> Remotefiles --> FreeStyle --> OK

Build Steps --> Send files or execute commands over SSH -->

Name = autoselect

Build now --> First build empty job because it will create a directory in workspace

```
cd /var/lib/jenkins/workspace/remotefiles/
mkdir files
touch test.py
```

Sourcefiles = files/test.py or top copy multiple files files/\*

Remove Prefix = empty [if you want to remote prefix directory and copy only files]

Remote Directory = empty [you can specify remote server directory to copy]

Exec command = Empty [if you want to execute file, sh  
/home/ec2-user/files/bash.sh]

Advance --> Exclude files if you want -->Exclude files --> files/test1.py,  
files/test2.py

Build now

See the files in remote server --> /home/ec2-user/files/

```
=====
=====
```

Variables , Jenkins admin activities, PASSWORDLESS LOGIN, BUILD Executors

```
=====
===
```

Setup Jenkins using shell script --> Jenkins.sh  
sh Jenkins.sh

Restore Deleted Job  
=====

If you accidentally delete the job , how to restore

Install plugin = job configuration history

Delete a job now --> Go to Dashboard --> click on job config history --> show  
delete jobs --> restore

note: If you install the plugin initially then all jobs create after plugin  
installation , restore option is available

Create a new job and delete

Delete a job now --> Go to Dashboard --> click on job config history --> show  
delete jobs --> restore

```
=====
VARIABLES
=====
```

VARIABLES: are used to store values, changes frequently : ex: Name = Reyaz

1. User Defined Variables 2. Jenkins Env Variables

1. USER DEFINED VARS: these variables will be declared by user.

--> LOCAL VARS: these variables will work inside job, works for only one job

--> GLOBAL VARS: these variables will work for all jobs in Jenkins

2. JENKINS ENV VARIABLES: these variables will be given by Jenkins.  
user cant define Jenkins variables.

Example for Local variable :

=====

Create a new job --> Free Style --> Build Step --> Execute shell (no git required)

echo "This is DevOps Course, here you are learning DevOps, DevOps is good "

--> Here every time you use DevOps , instead mention it as a Variable course = devops

course=DevOps --> no space

echo "This is \$course Course, here you are learning \$course, \$course is good "

save and build now

and configure build again and change course variable as AWS and build now

Example for Global variable :

=====

Now remove, course=DevOps in build and build now, Jenkins will not understand

Now set Global Variables

Dashboard -> Manage Jenkins --> System --> Global properties --> Environment variables --> ADD --> name = course, value = DevOps

Example for Jenkins Env variables :

=====

Create a new Job --> Free Style --> Build Step --> Execute shell (no git required)

click on list of available environment variables

echo "The Current Build Number is \$BUILD\_NUMBER, Job Name is \$JOB\_NAME"

Build now and see the output

if you want all env to print use in shell : printenv and run the build

=====

JENKINS Port Change:

=====

TO search a file -- find / -name Jenkins.service

vi /usr/lib/systemd/system/jenkins.service

line 72 (8080=8069)

systemctl daemon-reload

systemctl restart jenkins.service

=====

Jenkins without Password Login

=====

find / -name config.xml

vi /var/lib/jenkins/config.xml (true=false)

<useSecurity>true</useSecurity> ---> False

systemctl restart jenkins.service

Put IP in browser and directly it will login without asking username and password

=====

BUILD Executors - Builds running parallelly

=====

By default Jenkins runs the jobs sequentially --> click on the build 2 times , it will run only 2 at a time

Manage Jenkins --> System --> # of executors = 2 by default

if you want to run parallelly --> Job --> configure --> Execute concurrent builds if necessary --> build now (now run parallel)

or

Go to Dashboard --> Click on Build Executors Status-->Built-In Node--> Configure --> Number of executors --> 10